

LAPORAN PRAKTIKUM 1

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Fagil Lola Prihernando (2410651031)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

LATAR BELAKANG

Sistem operasi merupakan komponen utama dalam suatu komputer yang berperan sebagai penghubung antara pengguna dengan perangkat keras. Tanpa sistem operasi, pengguna tidak dapat menjalankan program maupun mengelola sumber daya komputer secara efisien. Melalui sistem operasi, berbagai proses seperti manajemen memori, pengendalian perangkat, pengaturan berkas, serta komunikasi antarproses dapat berjalan dengan teratur dan aman. Salah satu sistem operasi yang banyak digunakan di lingkungan akademik maupun profesional adalah Linux, karena bersifat open source, stabil, dan mendukung eksplorasi langsung terhadap struktur internal sistem. Dalam praktikum ini, mahasiswa mempelajari berbagai aspek penting dari sistem operasi, mulai dari layanan dasar (Operating-System Services), antarmuka pengguna (User and OS Interface), System Calls, hingga proses pembuatan dan pengelolaan System Services.

Selain itu, praktikum juga membahas mekanisme Linkers and Loaders, struktur sistem operasi, proses booting, serta debugging pada level kernel. Melalui kegiatan ini, mahasiswa diharapkan tidak hanya memahami teori, tetapi juga mampu menerapkan konsep-konsep sistem operasi secara langsung melalui eksperimen dan pemrograman di lingkungan Linux, sehingga memiliki pemahaman mendalam tentang cara kerja dan desain sistem operasi modern.

RUMUSAN MASALAH

1. Bagaimana sistem operasi Linux memberikan layanan dasar seperti manajemen proses, memori, dan sistem berkas melalui Operating-System Services dan System Calls?
2. Bagaimana pengguna dan program dapat berinteraksi dengan sistem operasi melalui antarmuka Command Line Interface (CLI) dan mekanisme systemd service?
3. Bagaimana proses pembuatan, pemuatan, serta pengujian komponen sistem operasi seperti kernel module, linkers, dan loaders dilakukan untuk memahami struktur internal sistem operasi?

TUJUAN

1. Memahami konsep dan fungsi utama sistem operasi dalam mengelola sumber daya komputer serta memberikan layanan kepada pengguna dan program.

2. Mempelajari dan mempraktikkan berbagai mekanisme interaksi dengan sistem operasi Linux, termasuk penggunaan perintah, pemrograman system call, serta pembuatan daemon service.
3. Menguasai proses implementasi dan analisis komponen sistem operasi seperti kernel module, linkers, loaders, serta memahami alur booting dan debugging pada sistem Linux.

BAB II

PEMBAHASAN

2.1 Operating-System Services

Operating-System Services pada Linux merupakan sekumpulan fungsi penting yang memungkinkan sistem menjalankan dan mengelola program dengan efisien. Melalui layanan seperti eksekusi program, operasi input-output, manipulasi sistem file, komunikasi antar proses, deteksi kesalahan, serta alokasi sumber daya, sistem operasi memastikan stabilitas dan kinerja komputer tetap optimal. Perintah seperti `uname -a`, `ps aux`, `free -h`, dan `lscpu` membantu pengguna memantau kondisi sistem, sementara program seperti `os_services.c` menunjukkan bagaimana layanan OS bekerja secara langsung, misalnya dalam menulis file, membaca informasi proses, dan mengatur penggunaan memori.

2.1.1 Menjelajahi Layanan OS

- Lihat informasi sistem

```
asus@LAPTOP-VRS206DG:~$ uname -a
Linux LAPTOP-VRS206DG 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
asus@LAPTOP-VRS206DG:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:        24.04
Codename:       noble
asus@LAPTOP-VRS206DG:~$ |
```

- Lihat proses yang berjalan

```
asus@LAPTOP-VRS206DG:~$ ps aux | head -20
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.4  0.1 21844 11760 ?        Ss   20:40   0:01 /sbin/init
root         2  0.0  0.0   3972  1664 ?        Sl   20:40   0:00 /init
root         7  0.0  0.0   3972  1792 ?        Sl   20:40   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root        47  1.3  0.1 66820 15120 ?        S<s  20:40   0:04 /usr/lib/systemd/systemd-journald
root        94  0.0  0.0  25108  6144 ?        Ss   20:40   0:00 /usr/lib/systemd/systemd-udevd
systemd+  109  0.0  0.1 21456 12032 ?        Ss   20:40   0:00 /usr/lib/systemd/systemd-resolved
systemd+  117  0.0  0.0  91024  7552 ?        Ssl  20:40   0:00 /usr/lib/systemd/systemd-timesyncd
root       168  0.0  0.0   4236  2688 ?        Ss   20:40   0:00 /usr/sbin/cron -f -P
message+  169  0.0  0.0   9532  4736 ?        Ss   20:40   0:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --sysl
og-only
root       183  0.0  0.1  17968  8320 ?        Ss   20:40   0:00 /usr/lib/systemd/systemd-logind
root       186  0.0  0.1 1756096 12416 ?       Ssl  20:40   0:00 /usr/libexec/wsl-pro-service -vv
syslog    200  0.0  0.0 222508  5632 ?        Ssl  20:40   0:00 /usr/sbin/rsyslogd -n -iNONE
root     205  0.0  0.0   3160  1920 hvcc0 Ss+  20:40   0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
root     208  0.0  0.0   3116  1792 tty1 Ss+  20:40   0:00 /sbin/agetty -o -p -- \u --noclear - linux
root     214  0.0  0.2 107032 22144 ?       Ssl  20:40   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-si
gnal
root     286  0.0  0.0   3076  1024 ?        Ss   20:40   0:00 /init
root     288  0.0  0.0   3092  1028 ?        S   20:40   0:00 /init
asus     296  0.0  0.0   6068  5248 pts/0 Ss   20:40   0:00 -bash
root     298  0.0  0.0   6824  3968 pts/1 Ss   20:40   0:00 /bin/login -f
asus@LAPTOP-VRS206DG:~$ |
```

- Lihat penggunaan memori

```
asus@LAPTOP-VRS206DG:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.4Gi         522Mi         6.3Gi         3.5Mi         781Mi         6.9Gi
Swap:          2.0Gi           0B          2.0Gi
asus@LAPTOP-VRS206DG:~$ |
```

User and Operating-System Interface pada Linux adalah jembatan interaksi antara pengguna dan sistem operasi melalui Command Line Interface (CLI). Pengguna dapat menavigasi sistem menggunakan perintah seperti `pwd`, `ls`, `cd`, dan `mkdir`, serta mengelola data dengan `pipelining`

dan redirection. Selain itu, Linux juga mendukung proses berjalan di latar belakang menggunakan `&`, `jobs`, dan `fg`. Dalam praktik program `simple_shell.c`, konsep ini ditunjukkan melalui pembuatan shell sederhana yang menggunakan `fork()` dan `execvp()` untuk mengeksekusi perintah pengguna, sehingga menggambarkan bagaimana sistem operasi menangani proses dan komunikasi antar program secara langsung.

2.2.1 Command Line Interface (CLI)

- Navigasi dasar

```
memory allocated successfully
asus@LAPTOP-VRS206DG:~$ pwd
/home/asus
asus@LAPTOP-VRS206DG:~$ ls -la
total 276
drwxr-xr-x 5 asus asus 4096 Oct 15 20:52 .
drwxr-xr-x 3 root root 4096 Oct 7 21:23 ..
-rw-r--r-- 1 asus asus 6047 Oct 14 11:25 .bash_history
-rw-r--r-- 1 asus asus 220 Oct 7 21:23 .bash_logout
-rw-r--r-- 1 asus asus 3771 Oct 7 21:23 .bashrc
drwx----- 3 asus asus 4096 Oct 9 17:16 .cache
drwx----- 3 asus asus 4096 Oct 9 16:23 .config
-rw-r--r-- 1 asus asus 20 Oct 9 16:52 .lessht
drwxr-xr-x 3 asus asus 4096 Oct 7 21:28 .local
-rw-rw-r-- 1 asus asus 0 Oct 15 20:40 .motd_shown
-rw-r--r-- 1 asus asus 807 Oct 7 21:23 .profile
-rw-r--r-- 1 asus asus 0 Oct 7 22:04 .sudo_as_admin_successful
-rwxr-xr-x 1 asus asus 17776 Oct 9 17:10 buggy
-rw-r--r-- 1 asus asus 332 Oct 9 17:10 buggy.c
-rwxr-xr-x 1 asus asus 17176 Oct 11 13:26 file_monitor
-rw-r--r-- 1 asus asus 5210 Oct 14 11:20 file_monitor.c
-rwxr-xr-x 1 asus asus 16352 Oct 7 22:48 file_syscalls
-rw-r--r-- 1 asus asus 966 Oct 7 22:46 file_syscalls.c
-rwxr-xr-x 1 asus asus 16384 Oct 7 22:49 info_syscalls
-rw-r--r-- 1 asus asus 711 Oct 7 22:48 info_syscalls.c
-rw-r--r-- 1 asus asus 1 Oct 7 22:47 info_syscalls.c:
-rw-r--r-- 1 asus asus 0 Oct 11 13:27 log.txt
-rwxr-xr-x 1 asus asus 16488 Oct 15 20:52 os_services
-rw-r--r-- 1 asus asus 1428 Oct 9 13:18 os_services.c
-rw-r--r-- 1 asus asus 0 Oct 9 13:15 os_services.c:
-rwxr-xr-x 1 asus asus 22208 Oct 9 19:43 process_menanger
-rw-r--r-- 1 asus asus 14751 Oct 9 19:42 process_menanger.c
-rwxr-xr-x 1 asus asus 16360 Oct 7 22:48 process_syscalls
-rw-r--r-- 1 asus asus 831 Oct 7 22:44 process_syscalls.c
-rw-r--r-- 1 asus asus 908 Oct 7 22:40 process_syscalls.c:
-rw-r--r-- 1 asus asus 1 Oct 7 22:41 process_syscalls.c:.save
-rwxr-xr-x 1 asus asus 16528 Oct 7 22:33 simple_shell
-rw-r--r-- 1 asus asus 908 Oct 7 22:33 simple_shell.c
-rw-r--r-- 1 asus asus 25 Oct 7 22:54 syscall_test.txt
-rw-r--r-- 1 asus asus 23 Oct 15 20:53 test_file.txt
-rw-r--r-- 1 asus asus 10938 Oct 9 17:01 trace.txt
asus@LAPTOP-VRS206DG:~$ cd /tmp
asus@LAPTOP-VRS206DG:/tmp$ mkdir test_dir
asus@LAPTOP-VRS206DG:/tmp$ cd test_dir
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Pipelining

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ls -l | grep "^d"
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ps aux | grep bash | wc -l
3
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Redirection

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ echo "Hello World" > output.txt
asus@LAPTOP-VRS206DG:/tmp/test_dir$ cat output.txt
Hello World
asus@LAPTOP-VRS206DG:/tmp/test_dir$ cat output.txt >> output.txt
cat: output.txt: input file is output file
asus@LAPTOP-VRS206DG:/tmp/test_dir$ cat < output.txt
Hello World
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- **Background processes**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ sleep 30 &
[1] 1098
asus@LAPTOP-VRS206DG:/tmp/test_dir$ jobs
[1]+  Running                  sleep 30 &
asus@LAPTOP-VRS206DG:/tmp/test_dir$ fg %1
sleep 30
```

2.2.2 Program C - Shell Sederhana

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nano simple_shell.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -o simple_shell simple_shell.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ./simple_shell
myshell> echo fagil
fagil
myshell> ^Z
[1]+  Stopped                  ./simple_shell
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

2.3 System Calls

System Calls merupakan mekanisme penting yang memungkinkan program aplikasi berinteraksi langsung dengan kernel sistem operasi. Melalui system call, program dapat meminta layanan seperti pembuatan proses, manajemen file, maupun pemeliharaan informasi sistem. Dalam praktik 3.1, **Process Control System Calls** seperti `fork()`, `exec()`, dan `wait()` digunakan untuk membuat proses baru, mengeksekusi program lain, serta menunggu proses anak selesai. Pada praktik 3.2, **File Management System Calls** seperti `open()`, `write()`, `read()`, `close()`, dan `unlink()` menunjukkan bagaimana program dapat membuat, menulis, membaca, menutup, dan menghapus file secara langsung melalui kernel. Sementara itu, praktik 3.3 menggunakan **Information Maintenance System Calls** seperti `getpid()`, `getuid()`, `time()`, dan `times()` untuk menampilkan informasi proses, pengguna, waktu sistem, serta aktivitas CPU. Dengan demikian, system calls berperan penting dalam menjembatani komunikasi antara aplikasi pengguna dan sumber daya inti sistem operasi Linux.

2.3.1 Process Control System Calls

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nano process_syscalls.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -o process_syscalls process_syscalls.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ./process_syscalls
=== Process Control System Calls ===
Parent PID: 1657
Parent waiting for child (PID: 1658)...
Child PID: 1658
Child's Parent PID: 1657
Child executing 'ls' command...
total 92
-rwxr-xr-x 1 asus asus 16352 Oct 15 21:23 file_syscalls
-rw-r--r-- 1 asus asus 966 Oct 15 21:15 file_syscalls.c
-rwxr-xr-x 1 asus asus 16384 Oct 15 21:23 info_syscalls
-rw-r--r-- 1 asus asus 711 Oct 15 21:20 info_syscalls.c
-rw-r--r-- 1 asus asus 12 Oct 15 21:02 output.txt
-rwxr-xr-x 1 asus asus 16360 Oct 15 21:26 process_syscalls
-rw-r--r-- 1 asus asus 831 Oct 15 21:14 process_syscalls.c
-rwxr-xr-x 1 asus asus 16528 Oct 15 21:09 simple_shell
-rw-r--r-- 1 asus asus 890 Oct 15 21:08 simple_shell.c
-rw-r--r-- 1 asus asus 25 Oct 15 21:24 syscall_test.txt
Child exited with status: 0
```

2.3.2 File Management System Calls

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nano file_syscalls.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -o file_syscalls file_syscalls.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ./file_syscalls
=== File Management System Calls ===
File descriptor: 3
Bytes written: 25
File closed
Read from file: Hello from system calls!
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

2.3.3 Information Maintenance System Calls

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nano info_syscalls.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -o file_syscalls file_syscalls.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ./info_syscalls
=== Information Maintenance System Calls ===
Process ID: 1740
Parent Process ID: 296
User ID: 1000
Group ID: 1000
Current time: Wed Oct 15 21:29:38 2025
Clock ticks since boot: 1718256038
Sleeping for 2 seconds...
Awake!
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

2.4 System Services

System Services di Linux merupakan komponen penting yang berjalan di latar belakang untuk memastikan sistem dan aplikasi berfungsi dengan baik. Layanan ini dikelola oleh systemd, yaitu sistem init modern yang bertanggung jawab untuk mengatur proses booting, manajemen service, dan kontrol proses daemon. Dalam praktik 4.1, pengguna dapat melihat semua layanan aktif dengan perintah `systemctl list-units --type=service`, memeriksa status layanan seperti ssh menggunakan `systemctl status ssh`, serta meninjau log aktivitas melalui `journalctl -u ssh`. Selain itu, `systemctl list-unit-files --type=service --state=enabled` digunakan untuk mengetahui service yang otomatis dijalankan saat boot.

Pada praktik 4.2, pengguna mempelajari cara membuat custom system service dengan program daemon sederhana bernama *my_daemon.c*. Program ini dijalankan di latar belakang menggunakan mekanisme `fork()`, `setsid()`, dan `syslog()` untuk mencatat aktivitas ke log sistem. File konfigurasi service kemudian dibuat di `/etc/systemd/system/my_daemon.service`, yang menentukan bagaimana daemon dijalankan dan dikelola oleh systemd. Dengan menggunakan perintah `systemctl`, pengguna dapat memulai, menghentikan, memantau, dan mengatur agar service berjalan otomatis saat startup. Melalui praktik ini, dipahami bahwa systemd dan system services berperan penting dalam menjaga stabilitas, otomatisasi, dan pengelolaan proses di sistem operasi Linux secara efisien dan terstruktur.

- **Lihat semua service**

- **Status service**

- **Lihat log service**

- **Lihat service yang berjalan saat boot**

- ```
anous@LAPTOP-VRQ2869G:/tmp/test_dir$ systemctl list-unit-files --type=service --state=enabled
```
- | UNIT FILE                                | STATE   | PRESET  |
|------------------------------------------|---------|---------|
| apparmor.service                         | enabled | enabled |
| apport.service                           | enabled | enabled |
| cloud-config.service                     | enabled | enabled |
| cloud-final.service                      | enabled | enabled |
| cloud-init-local.service                 | enabled | enabled |
| cloud-init.service                       | enabled | enabled |
| console-setup.service                    | enabled | enabled |
| cron.service                             | enabled | enabled |
| dmesg.service                            | enabled | enabled |
| e2scrub_reap.service                     | enabled | enabled |
| getty@.service                           | enabled | enabled |
| keyboard-setup.service                   | enabled | enabled |
| landscape-client.service                 | enabled | enabled |
| networkd-dispatcher.service              | enabled | enabled |
| rsyslog.service                          | enabled | enabled |
| setvtrgb.service                         | enabled | enabled |
| snapped.apparmor.service                 | enabled | enabled |
| snapped.autoimport.service               | enabled | enabled |
| snapped.core-fixup.service               | enabled | enabled |
| snapped.recovery-chooser-trigger.service | enabled | enabled |
| snapped.seeded.service                   | enabled | enabled |
| snapped.service                          | enabled | enabled |
| snapped.system-shutdown.service          | enabled | enabled |
| systemd-pstore.service                   | enabled | enabled |
| systemd-resolved.service                 | enabled | enabled |
| systemd-timesyncd.service                | enabled | enabled |
| ua-reboot-cmds.service                   | enabled | enabled |
| ubuntu-advantage.service                 | enabled | enabled |
| unattended-upgrades.service              | enabled | enabled |
| wsl-pro.service                          | enabled | enabled |
- 30 unit files listed.

## 2.4.2 Membuat Custom System Service

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nano my_daemon.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -o my_daemon my_daemon.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ sudo nano /etc/systemd/system/my_daemon.service
asus@LAPTOP-VRS206DG:/tmp/test_dir$ sudo systemctl daemon-reload
asus@LAPTOP-VRS206DG:/tmp/test_dir$ sudo systemctl start my_daemon
Job for my_daemon.service failed because the control process exited with error code.
See "systemctl status my_daemon.service" and "journalctl -xeu my_daemon.service" for details.
asus@LAPTOP-VRS206DG:/tmp/test_dir$ sudo systemctl status my_daemon
* my_daemon.service - My Custom Daemon
 Loaded: loaded (/etc/systemd/system/my_daemon.service; disabled; preset: enabled)
 Active: failed (Result: exit-code) since Wed 2025-10-15 22:06:57 WIB; 1min 6s ago
 Process: 2784 ExecStart=/path/to/my_daemon (code=exited, status=203/EXEC)
 CPU: 3ms

Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 5.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Start request repeated too quickly.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
asus@LAPTOP-VRS206DG:/tmp/test_dir$ sudo journalctl -u my_daemon
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 1.
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 2.
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 15 22:06:56 LAPTOP-VRS206DG systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 3.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 4.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 5.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Start request repeated too quickly.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 15 22:06:57 LAPTOP-VRS206DG systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
asus@LAPTOP-VRS206DG:/tmp/test_dir$ sudo systemctl stop my_daemon
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

## 2.5 Linkers and Loaders

Dalam sistem operasi Linux, *linkers* dan *loaders* memiliki peran penting dalam proses pembuatan dan eksekusi program. Linker bertugas menggabungkan berbagai *object file* hasil kompilasi (seperti `main.o` dan `math_ops.o`) menjadi satu *executable file* yang utuh, misalnya program. Pada proses ini, linker menyelesaikan referensi antar file, seperti pemanggilan fungsi `add()` dan `multiply()` yang didefinisikan di file lain. Loader, di sisi lain, bertugas memuat program tersebut ke dalam memori saat dijalankan, menyiapkan ruang memori, serta menghubungkan dependensi *shared library* bila menggunakan *dynamic linking*. Dalam praktik 5.1, proses linking dilakukan setelah tahap preprocessing, compiling, dan assembling menggunakan perintah `gcc main.o math_ops.o -o program`. Hasilnya dapat dianalisis dengan `nm` untuk melihat simbol, dan `ldd` untuk melihat dependensi. Melalui praktik ini, mahasiswa memahami perbedaan antara *static* dan *dynamic linking*, serta bagaimana membuat *shared library* (`libmathops.so`) yang dapat digunakan berulang kali tanpa perlu dikompilasi ulang.

## 2.5.1 Proses Lingking

- Buat file math\_ops.h

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nano math_ops.h
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Buat file math\_ops.c

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nano math_ops.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Buat main .c

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nano main.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Preprocessing

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -E main.c -o main.i
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -E math_ops.c -o math_ops.i
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Compilation (assembly)

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -S main.c -o main.s
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -S math_ops.c -o math_ops.s
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Assembly (object files)

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -c main.c -o main.o
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -c math_ops.c -o math_ops.o
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Linking

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc main.o math_ops.o -o program
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Lihat symbols dalam object file

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nm main.o
0000000000000000 U add
0000000000000000 T main
0000000000000000 U multiply
0000000000000000 U printf
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nm math_ops.o
0000000000000000 T add
0000000000000000 T multiply
asus@LAPTOP-VRS206DG:/tmp/test_dir$ nm program
0000000000003dc8 d _DYNAMIC
0000000000003f58 d _GLOBAL_OFFSET_TABLE_
0000000000002000 R _IO_stdin_used
w _ITM_deregisterTMCloneTable
w _ITM_registerTMCloneTable
00000000000002148 r __FRAME_END__
000000000000201c r __GNU_EH_FRAME_HDR
00000000000004010 D __TMC_END__
0000000000000038c r __abi_tag
00000000000004010 b __bss_start
w _cxa_finalize@GLIBC_2.2.5
00000000000004000 D __data_start
00000000000001100 t __do_global_ctors_aux
0000000000003dc0 d __do_global_ctors_aux_fini_array_entry
00000000000004008 D __dso_handle
0000000000003db5 d __frame_dummy_init_array_entry
w __mon_start__
U __libc_start_main@GLIBC_2.34
00000000000004010 D _edata
00000000000004010 b _end
000000000000011d4 t _fini
00000000000001000 T _init
00000000000001000 T _start
000000000000011a2 T add
00000000000004010 b completed.0
00000000000004000 w data_start
00000000000001000 t deregister_tm_clones
00000000000001140 t frame_dummy
00000000000001140 T main
000000000000011ba T multiply
000000000000018c0 t register_tm_clones
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- Lihat dependencies

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ldd program
linux-vdso.so.1 (0x00007ffff3c849000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000075d0efa00000)
/lib64/ld-linux-x86-64.so.2 (0x000075d0efc2f000)
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

## 2.5.2 Static vs Dynamic Linking

- Static linking

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc main.c math_ops.c -o program_static -static
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Dynamic linking (default)

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc main.c math_ops.c -o program_dynamic
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Bandingkan ukuran

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ls -lh program_static program_dynamic
-rwxr-xr-x 1 asus asus 16K Oct 15 22:34 program_dynamic
-rwxr-xr-x 1 asus asus 767K Oct 15 22:32 program_static
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Lihat dependencies

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ldd program_static
not a dynamic executable
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ldd program_dynamic
linux-vdso.so.1 (0x00007ffff1cf7000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007890b4a00000)
/lib64/ld-linux-x86-64.so.2 (0x00007890b4d97000)
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

## 2.5.3 Membuat dan Menggunakan Shared Library

- Compile sebagai shared library

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -c -fPIC math_ops.c -o math_ops.o
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc -shared -o libmathops.so math_ops.o
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Compile program yang menggunakan library

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ gcc main.c -L. -lmathops -o program_shared
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Jalankan dengan LD\_LIBRARY\_PATH

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ LD_LIBRARY_PATH=. ./program_shared
5 + 3 = 8
5 * 3 = 15
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Lihat informasi loader

```

asus@LAPTOP-VRS206DG:/tmp/test_dir$ readelf -d program_shared
Dynamic section at offset 0x2da8 contains 28 entries:
 Tag Type Name/Value
 0x0000000000000001 (NEEDED) Shared library: [libmathops.so]
 0x0000000000000001 (NEEDED) Shared library: [libc.so.6]
 0x000000000000000c (INIT) 0x1000
 0x000000000000000d (FINI) 0x11e4
 0x0000000000000019 (INIT_ARRAY) 0x3d98
 0x000000000000001b (INIT_ARRAYSZ) 8 (bytes)
 0x000000000000001a (FINI_ARRAY) 0x3da0
 0x000000000000001c (FINI_ARRAYSZ) 8 (bytes)
 0x000000006ffffff5 (GNU_HASH) 0x3b0
 0x0000000000000005 (STRTAB) 0x4b0
 0x0000000000000006 (SYMTAB) 0x3d8
 0x000000000000000a (STRSZ) 170 (bytes)
 0x000000000000000b (SYMMENT) 24 (bytes)
 0x0000000000000015 (DEBUG) 0x0
 0x0000000000000003 (PLTGOT) 0x3fa8
 0x0000000000000002 (PLTRELSZ) 72 (bytes)
 0x0000000000000014 (PLTREL) REL
 0x0000000000000017 (JMPREL) 0x660
 0x0000000000000007 (RELA) 0x5a0
 0x0000000000000008 (RELASZ) 192 (bytes)
 0x0000000000000009 (RELAENT) 24 (bytes)
 0x000000000000001e (FLAGS) BIND_NOW
 0x000000006ffffffb (FLAGS_1) Flags: NOW PIE
 0x000000006ffffffe (VERNEED) 0x570
 0x000000006ffffffc (VERNEEDNUM) 1
 0x000000006ffffff0 (VERSYM) 0x55a
 0x000000006ffffff9 (RELACOUNT) 3
 0x0000000000000000 (NULL) 0x0
asus@LAPTOP-VRS206DG:/tmp/test_dir$

```

## 2.6 Why Applications Are Operating-System Specific

Aplikasi bersifat *operating-system specific* karena setiap sistem operasi memiliki arsitektur internal, format biner, serta sistem pemanggilan (*system calls*) yang berbeda. *System call* adalah antarmuka utama antara aplikasi dan kernel, yang mengatur komunikasi seperti manajemen proses, file, dan perangkat keras. Misalnya, pada Linux fungsi `getpid()` digunakan untuk mendapatkan ID proses melalui pustaka `<unistd.h>`, sedangkan di Windows fungsi serupa membutuhkan pustaka dan API berbeda seperti `<windows.h>`.

Pada Praktik 6.1, file `os_specific.c` menunjukkan perbedaan ini dengan menggunakan *directive preprocessor* (`#ifdef _linux_`, `#elif _WIN32`) untuk menyesuaikan kode berdasarkan sistem operasi yang digunakan. Program tersebut mencetak informasi OS dengan fungsi `uname()` pada Linux atau menggunakan API Windows ketika dijalankan di sistem Microsoft.

Selanjutnya, Praktik 6.2 menganalisis *binary format* hasil kompilasi menggunakan perintah seperti `file`, `readelf`, dan `objdump`. Di Linux, executable berformat ELF (Executable and Linkable Format), sedangkan Windows menggunakan PE (Portable Executable). Perbedaan format ini mempertegas bahwa program yang dikompilasi di satu sistem operasi tidak dapat langsung dijalankan di OS lain tanpa *recompilation* atau *compatibility layer* seperti Wine.

### 2.6.1 System Call Differences

```

asus@LAPTOP-VRS206DG:/tmp/test_dir$ nano os_specific.c
asus@LAPTOP-VRS206DG:/tmp/test_dir$

```

## 2.6.2 Binary Format Analysis

- Lihat format executable

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ file program
program: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=8c5145e6b63b47095af229368edc342f23076fc2, for GNU/Linux 3.2.0, not stripped
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Lihat header ELF

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ readelf -h program
ELF Header:
 Magic: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
 Class: ELF64
 Data: 2's complement, little endian
 Version: 1 (current)
 OS/ABI: UNIX - System V
 ABI Version:
 0
 Type: DYN (Position-Independent Executable file)
 Machine: Advanced Micro Devices X86-64
 Version: 0x1
 Entry point address: 0x1060
 Start of program headers: 64 (bytes into file)
 Start of section headers: 14072 (bytes into file)
 Flags: 0x0
 Size of this header: 64 (bytes)
 Size of program headers: 56 (bytes)
 Number of program headers: 13
 Size of section headers: 64 (bytes)
 Number of section headers: 31
 Section header string table index: 30
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Lihat section headers

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ readelf -S program
There are 31 section headers, starting at offset 0x36f8:

Section Headers:
 [Nr] Name Type Address Offset
 Size EntSize Flags Link Info Align

 [0] NULL 0000000000000000 0 0
 0000000000000000
 [1] .interp PROGBITS 0000000000000118 00000118
 0000000000000000 A 0 0
 [2] .note.gnu.pr[...] NOTE 0000000000000138 00000138
 0000000000000000 A 0 0
 [3] .note.gnu.bu[...] NOTE 0000000000000368 00000368
 0000000000000000 A 0 0
 [4] .note.ABI-tag NOTE 000000000000038c 0000038c
 0000000000000020 0000000000000000 A 0 0
 [5] .gnu.hash GNU_HASH 00000000000003a8 000003a8
 0000000000000020 0000000000000000 A 6 0
 [6] .dynamic DYNAMIC 00000000000003d8 000003d8
 00000000000000a8 0000000000000118 A 7 1
 [7] .dynstr STRTAB 0000000000000400 00000400
 000000000000006f 0000000000000000 A 0 0
 [8] .gnu.version VERSYM 0000000000000510 00000510
 0000000000000000 0000000000000002 A 0 0
 [9] .gnu.version_r VERNEED 0000000000000520 00000520
 0000000000000030 0000000000000000 A 7 1
[10] .rela.dyn RELA 0000000000000550 00000550
 0000000000000060 0000000000000018 A 6 0
[11] .rela.plt RELA 0000000000000568 00000568
 0000000000000018 0000000000000118 A 6 24
[12] .init PROGBITS 0000000000001000 00001000
 0000000000000010 0000000000000000 AX 0 0
[13] .plt PROGBITS 0000000000001020 00001020
 0000000000000020 0000000000000010 AX 0 0
[14] .plt.got PROGBITS 0000000000001040 00001040
 0000000000000010 0000000000000010 AX 0 0
[15] .plt.sec PROGBITS 0000000000001050 00001050
 0000000000000010 0000000000000010 AX 0 0
[16] .text PROGBITS 0000000000001060 00001060
 0000000000000010 0000000000000000 AX 0 0
[17] .fini PROGBITS 0000000000001100 00001100
 0000000000000000 0000000000000000 AX 0 0
[18] .rodata PROGBITS 0000000000002000 00002000
 0000000000000000 0000000000000000 A 0 0
[19] .eh_frame_hdr PROGBITS 000000000000201c 0000201c
 0000000000000000 0000000000000000 A 0 0
[20] .eh_frame PROGBITS 0000000000002060 00002060
 0000000000000000 0000000000000000 A 0 0
[21] .init_array INIT_ARRAY 0000000000003d80 00003d80
 0000000000000000 0000000000000000 WA 0 0
[22] .fini_array FINI_ARRAY 0000000000003dc0 00003dc0
 0000000000000000 0000000000000000 WA 0 0
[23] .dynamic DYNAMIC 0000000000003dc0 00003dc0
 00000000000001f0 0000000000000110 WA 7 0
[24] .got PROGBITS 0000000000003fd0 00003fd0
 0000000000000040 0000000000000000 WA 0 0
[25] .data PROGBITS 0000000000004000 00004000
 0000000000000010 0000000000000000 WA 0 0
[26] .bss UNINIT 0000000000004010 00004010
 0000000000000000 0000000000000000 WA 0 0
[27] .comment PROGBITS 0000000000004010 00004010
 0000000000000020 0000000000000001 RS 0 0
[28] .symtab SYMTAB 0000000000004000 00004000
 00000000000003a8 0000000000000018 29 19 8
[29] .strtab STRTAB 0000000000004000 00004000
 00000000000001f0 0000000000000000 0 0 1
[30] .shstrtab STRTAB 0000000000004000 00004000
 000000000000011a 0000000000000000 0 0 1

Key to Flags:
 W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
 L (link order), O (extra OS processing required), G (group), T (TLS),
 C (compressed), x (unknown), o (OS specific), E (exclude),
 D (debug), l (large), p (processor specific)
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- **Lihat program headers**

```
asus@LAPTOP-VRS286DG:/tmp/test_dir$ readelf -l program
Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1000
There are 13 program headers, starting at offset 64

Program Headers:
Type Offset VirtAddr PhysAddr
 FileSiz MemSiz Flags Align
PHDR 0x0000000000000040 0x0000000000000040 0x0000000000000040
 0x0000000000002d8 0x0000000000002d8 R 0x8
INTERP 0x0000000000000318 0x0000000000000318 0x0000000000000318
 0x000000000000001c 0x000000000000001c R 0x1
 [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD 0x0000000000000000 0x0000000000000000 0x0000000000000000
 0x000000000000628 0x000000000000628 R 0x1000
LOAD 0x0000000000001000 0x0000000000001000 0x0000000000001000
 0x0000000000001e1 0x0000000000001e1 R E 0x1000
LOAD 0x0000000000002000 0x0000000000002000 0x0000000000002000
 0x00000000000014c 0x00000000000014c R 0x1000
LOAD 0x0000000000002db8 0x0000000000003db8 0x0000000000003db8
 0x000000000000258 0x000000000000260 RW 0x1000
DYNAMIC 0x0000000000002dc8 0x0000000000003dc8 0x0000000000003dc8
 0x0000000000001f0 0x0000000000001f0 RW 0x8
NOTE 0x000000000000338 0x000000000000338 0x000000000000338
 0x000000000000030 0x000000000000030 R 0x3
NOTE 0x000000000000368 0x000000000000368 0x000000000000368
 0x000000000000044 0x000000000000044 R 0x4
GNU_PROPERTY 0x000000000000338 0x000000000000338 0x000000000000338
 0x000000000000030 0x000000000000030 R 0x8
GNU_EH_FRAME 0x000000000000201c 0x000000000000201c 0x000000000000201c
 0x000000000000044 0x000000000000044 R 0x4
GNU_STACK 0x0000000000000000 0x0000000000000000 0x0000000000000000
 0x000000000000000 0x000000000000000 RW 0x10
GNU_RELRO 0x0000000000002db8 0x0000000000003db8 0x0000000000003db8
 0x000000000000248 0x000000000000248 R 0x1

Section to Segment mapping:
Segment Sections...
00
01 .interp
02 .interp.note.gnu.property.note.gnu.build-id.note.ABI-tag.gnu.hash.dynsym.dynstr.gnu.version.gnu.version_r.rela.dyn.rela.plt
03 .init.plt.plt.got.plt.sec.text.fini
04 .rodata.eh_frame_hdr.eh_frame
05 .init_array.fini_array.dynamic.got.data.bss
06 .dynamic
07 .note.gnu.property
08 .note.gnu.build-id.note.ABI-tag
09 .note.gnu.property
10 .eh_frame_hdr
11
12 .init_array.fini_array.dynamic.got
```

- **Disassemble**

```
asus@LAPTOP-VRS286DG:/tmp/test_dir$ objdump -d program | head -50
program: file format elf64-x86-64

Disassembly of section .init:

0000000000001000 <.init>:
1000: f3 0f 1e fa endbr64
1004: 48 83 ec 08 sub $0x8,%rsp
1008: 48 05 d9 2f 00 00 mov 0x2f9a(%rip),%rax # 3fe8 <__gmon_start__@Base>
100f: 48 85 c0 test %rax,%rax
1012: 74 02 je 1016 <.init+0x16>
1014: ff d0 call *%rax
1016: 48 83 c4 08 add $0x8,%rsp
101a: c3 ret

Disassembly of section .plt:

0000000000001020 <.plt>:
1020: ff 35 9a 2f 00 00 push 0x2f9a(%rip) # 3fc0 <_GLOBAL_OFFSET_TABLE_+0x8>
1026: ff 25 9c 2f 00 00 jmp *0x2f9c(%rip) # 3fc8 <_GLOBAL_OFFSET_TABLE_+0x10>
102c: 0f 1f 40 00 nopl 0x0(%rax)
1030: f3 0f 1e fa endbr64
1034: 68 00 00 00 00 push $0x0
1039: e9 c2 ff ff jmp 1020 <.init+0x20>
103e: 66 90 xchgb %ax,%ax

Disassembly of section .plt.got:

0000000000001040 <__cxa_finalize@plt>:
1040: f3 0f 1e fa endbr64
1044: ff 25 ae 2f 00 00 jmp *0x2fae(%rip) # 3ff8 <__cxa_finalize@GLIBC_2.2.5>
104a: 66 0f 1f 44 00 00 nopw 0x0(%rax,%rax,1)

Disassembly of section .plt.sec:

0000000000001050 <.printf@plt>:
1050: f3 0f 1e fa endbr64
1054: ff 25 76 2f 00 00 jmp *0x2f76(%rip) # 3fd0 <.printf@GLIBC_2.2.5>
105a: 66 0f 1f 44 00 00 nopw 0x0(%rax,%rax,1)

Disassembly of section .text:

0000000000001060 <.start>:
1060: f3 0f 1e fa endbr64
1064: 31 ed xor %ebp,%ebp
1066: 49 89 d1 mov %rdi,%r9
1069: 50 pop %rsi
106a: 48 89 e2 mov %rsp,%rdx
106d: 48 83 e4 f0 and $0xfffffffffffffff0,%rsp
asus@LAPTOP-VRS286DG:/tmp/test_dir$
```

## 2.7 OS Design and Implementation

Pada praktik ini, mahasiswa mempelajari bagaimana sistem operasi Linux dirancang dan diimplementasikan melalui **kernel module**. Kernel module merupakan potongan kode yang dapat dimuat (load) dan dilepas (unload) ke dalam kernel tanpa perlu melakukan kompilasi ulang seluruh kernel. Dalam file *hello\_module.c*, modul sederhana dibuat untuk menampilkan pesan “Loaded” saat dimasukkan dan “Unloaded” saat dilepas. Proses kompilasi dilakukan dengan *Makefile* menggunakan perintah *make*, lalu modul dimuat ke kernel menggunakan *insmod* dan dihapus dengan *rmmmod*. Melalui praktik ini, mahasiswa memahami konsep dasar bagaimana kernel Linux bekerja secara modular, serta cara sistem menangani pesan dan informasi melalui *kernel ring buffer* (*dmesg*). Selain itu, diperkenalkan pula cara memantau dan menganalisis informasi kernel menggunakan perintah seperti *uname -r*, *cat /proc/version*, *sysctl*, *lsmod*, dan *modinfo*. Praktik ini penting untuk memahami bagaimana komponen inti sistem operasi berinteraksi langsung dengan perangkat keras dan proses sistem.

### 2.7.1 Kernel Module Sederhana

```
asus@LAPTOP-VRS286DG:/tmp/test_dir$ nano hello_module.c
asus@LAPTOP-VRS286DG:/tmp/test_dir$
```

- **Makefile**

```
asus@LAPTOP-VRS286DG:/tmp/test_dir$ sudo nano Makefile
[sudo] password for asus:
asus@LAPTOP-VRS286DG:/tmp/test_dir$
```

- **Compile dan load**

```
asus@LAPTOP-VRS286DG:/tmp/test_dir$ make
Makefile:3: *** missing separator. Stop.
asus@LAPTOP-VRS286DG:/tmp/test_dir$ sudo insmod hello_module.ko
insmod: ERROR: could not load module hello_module.ko: No such file or directory
asus@LAPTOP-VRS286DG:/tmp/test_dir$ lsmod | grep hello
asus@LAPTOP-VRS286DG:/tmp/test_dir$ dmesg | tail
[8771.572324] systemd-journald[47]: Time jumped backwards, rotating.
[8821.976939] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[8826.653145] systemd-journald[47]: Time jumped backwards, rotating.
[8831.650343] systemd-journald[47]: Time jumped backwards, rotating.
[8881.806793] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[8886.757905] systemd-journald[47]: Time jumped backwards, rotating.
[8891.764632] systemd-journald[47]: Time jumped backwards, rotating.
[8941.841366] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[8946.853223] systemd-journald[47]: Time jumped backwards, rotating.
[8951.865047] systemd-journald[47]: Time jumped backwards, rotating.
asus@LAPTOP-VRS286DG:/tmp/test_dir$ sudo rmmmod hello_module
rmmmod: ERROR: Module hello_module is not currently loaded
asus@LAPTOP-VRS286DG:/tmp/test_dir$ dmesg | tail
[8831.650343] systemd-journald[47]: Time jumped backwards, rotating.
[8881.806793] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[8886.757905] systemd-journald[47]: Time jumped backwards, rotating.
[8891.764632] systemd-journald[47]: Time jumped backwards, rotating.
[8941.841366] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[8946.853223] systemd-journald[47]: Time jumped backwards, rotating.
[8951.865047] systemd-journald[47]: Time jumped backwards, rotating.
[9002.729400] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[9021.964437] systemd-journald[47]: Time jumped backwards, rotating.
[9027.002091] systemd-journald[47]: Time jumped backwards, rotating.
asus@LAPTOP-VRS286DG:/tmp/test_dir$
```



## 2.7.2 Monitoring Kernel

- Lihat informasi kernel

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ uname -r
6.6.87.2-microsoft-standard-WSL2
asus@LAPTOP-VRS206DG:/tmp/test_dir$ cat /proc/version
Linux version 6.6.87.2-microsoft-standard-WSL2 (root@439a258ad544) (gcc (GCC) 11.2.0, GNU ld (GNU Binutils) 2.37) #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Lihat parameter kernel

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ sysctl -a | head -20
sysctl: permission denied on key 'kernel.cad_pid'
abi.vsyscall32 = 1
debug.exception-trace = 1
debug.kprobes-optimization = 1
dev.cdrom.autoclose = 1
dev.cdrom.autoeject = 0
dev.cdrom.check_media = 0
dev.cdrom.debug = 0
dev.cdrom.info = CD-ROM information, Id: cdrom.c 3.20 2003/12/17
dev.cdrom.info =
dev.cdrom.info = drive name:
dev.cdrom.info = drive speed:
dev.cdrom.info = drive # of slots:
dev.cdrom.info = Can close tray:
dev.cdrom.info = Can open tray:
dev.cdrom.info = Can lock tray:
dev.cdrom.info = Can change speed:
dev.cdrom.info = Can select disk:
dev.cdrom.info = Can read multisession:
dev.cdrom.info = Can read MCN:
dev.cdrom.info = Reports media changed:
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- Lihat kernel ring buffer

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ dmesg | tail -30
[9823.178342] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[9828.175718] systemd-journald[47]: Time jumped backwards, rotating.
[9833.203904] systemd-journald[47]: Time jumped backwards, rotating.
[9883.220958] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[9888.247129] systemd-journald[47]: Time jumped backwards, rotating.
[9893.249936] systemd-journald[47]: Time jumped backwards, rotating.
[9943.465619] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 7 similar message(s))
[9948.319951] systemd-journald[47]: Time jumped backwards, rotating.
[9953.422977] systemd-journald[47]: Time jumped backwards, rotating.
[10008.453456] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 10 similar message(s))
[10013.512897] systemd-journald[47]: Time jumped backwards, rotating.
[10018.547297] systemd-journald[47]: Time jumped backwards, rotating.
[10068.846492] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[10073.574772] systemd-journald[47]: Time jumped backwards, rotating.
[10078.557074] systemd-journald[47]: Time jumped backwards, rotating.
[10128.669516] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[10134.153819] systemd-journald[47]: Time jumped backwards, rotating.
[10139.005304] systemd-journald[47]: Time jumped backwards, rotating.
[10189.029551] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[10193.721742] systemd-journald[47]: Time jumped backwards, rotating.
[10199.262184] systemd-journald[47]: Time jumped backwards, rotating.
[10248.892900] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[10253.893266] systemd-journald[47]: Time jumped backwards, rotating.
[10258.852766] systemd-journald[47]: Time jumped backwards, rotating.
[10308.909960] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[10313.924997] systemd-journald[47]: Time jumped backwards, rotating.
[10318.936531] systemd-journald[47]: Time jumped backwards, rotating.
[10369.015785] systemd-journald[47]: Time jumped backwards, rotating. (Dropped 9 similar message(s))
[10374.014892] systemd-journald[47]: Time jumped backwards, rotating.
[10379.025515] systemd-journald[47]: Time jumped backwards, rotating.
asus@LAPTOP-VRS206DG:/tmp/test_dir$
```

- **Lihat kernel modules**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ lsmod
Module Size Used by
tls 98304 0
intel_rapl_msr 16384 0
intel_rapl_common 32768 1 intel_rapl_msr
kvm_amd 126976 0
kvm 970752 1 kvm_amd
irqbypass 12288 1 kvm
crc32c_intel 16384 0
battery 20480 0
ac 16384 0
sch_fq_codel 16384 1
configfs 53248 1
autofs4 45056 0
br_netfilter 28672 0
bridge 282624 1 br_netfilter
stp 12288 1 bridge
llc 12288 2 bridge,stp
ip_tables 28672 0
tun 53248 0
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- **Informasi modul tertentu**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ modinfo ext4
name: ext4
filename: (builtin)
softdep: pre: crc32c
license: GPL
file: fs/ext4/ext4
description: Fourth Extended Filesystem
author: Remy Card, Stephen Tweedie, Andrew Morton, Andreas Dilger, Theodore Ts'o and others
alias: fs-ext4
alias: ext3
alias: fs-ext3
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

## 2.8 Operating-System Structure

Pada praktik ini, mahasiswa mempelajari struktur berlapis (*layering structure*) dari sistem operasi Linux serta konsep arsitektur kernel. Linux memiliki lima lapisan utama, yaitu Hardware, Kernel, System Calls, System Programs, dan User Applications. Lapisan hardware dieksplorasi menggunakan perintah seperti `lspci`, `lsusb`, dan `cat /proc/cpuinfo` untuk melihat perangkat yang terpasang. Lapisan kernel diperiksa melalui `uname -a`, sedangkan lapisan *system calls* diamati dengan `strace`, yang menampilkan panggilan sistem saat perintah dijalankan. Lapisan *system programs* seperti `ps`, `aux`, dan `top` menunjukkan proses aktif, sementara *user applications* seperti `bash` dan `gcc` mewakili aplikasi pengguna.

Pada praktik Microkernel vs Monolithic, mahasiswa membandingkan ukuran kernel Linux dan mempelajari perbedaan antara fitur *built-in* dan *modular* menggunakan `ls -lh`, `grep`, serta `lsmod`. Praktik ini menunjukkan bahwa Linux menggunakan pendekatan monolithic kernel, di mana sebagian besar fungsi dijalankan di ruang kernel.

Selanjutnya, dalam praktik Virtual File System (VFS), program `procfs_read.c` dibuat untuk membaca informasi sistem dari direktori `/proc`, seperti data CPU, memori, dan proses yang sedang berjalan. Hal ini menunjukkan bagaimana Linux menggunakan *virtual file system* untuk menyediakan antarmuka fleksibel antara kernel dan aplikasi pengguna tanpa langsung

mengakses perangkat keras. Melalui praktik ini, mahasiswa memahami bagaimana struktur sistem operasi diorganisasikan dan berinteraksi secara hierarkis dari level perangkat keras hingga aplikasi pengguna.

## 2.8.1 Eksplorasi Struktur Layering

- **Layer 1: Hardware**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ lspci
0000:00:00.0 System peripheral: Red Hat, Inc. Virtio file system (rev 01)
0000:00:00.0 3D controller: Microsoft Corporation Basic Render Driver
0000:00:00.0 SCSI storage controller: Red Hat, Inc. Virtio 1.0 console (rev 01)
asus@LAPTOP-VRS206DG:/tmp/test_dir$ lsusb
asus@LAPTOP-VRS206DG:/tmp/test_dir$ cat /proc/cpuinfo
processor : 0
vendor_id : AuthenticAMD
cpu family : 23
model : 160
model name : AMD Ryzen 5 7520U with Radeon Graphics
stepping : 0
microcode : 0xffffffff
cpu MHz : 2790.547
cache size : 512 KB
physical id : 0
siblings : 8
core id : 0
cpu cores : 4
apicid : 0
initial apicid : 0
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good nopl tsc_reliable nonstop_tsc cpuid extd_apicid tsc_known_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm cmp_legacy svm cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw
topoext perfctr_core ssbd lbrs ibpb stibp vmmcall fsgsbase bmi1 avx2 smep bmi2 rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 clzero xsaveerptr arat npt nrip_save tsc_scale vmcb_clean flushby
bugs : sysret_ss_attrs null_seg spectre_v1 spectre_v2 spec_store_bypass retbleed smt_rsb rsrso ibpb_no_ret
bogomips : 5589.69
tlb size : 3072 4K pages
clflush size : 64
cache_alignment : 64
address sizes : 48 bits physical, 48 bits virtual
power management:

processor : 1
vendor_id : AuthenticAMD
cpu family : 23
model : 160
model name : AMD Ryzen 5 7520U with Radeon Graphics
stepping : 0
microcode : 0xffffffff
cpu MHz : 2790.547
cache size : 512 KB
physical id : 0
siblings : 8
core id : 0
```

- **Layer 2: Kernel**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ uname -a
Linux LAPTOP-VRS206DG 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
asus@LAPTOP-VRS206DG:/tmp/test_dir$ cat /proc/sys/kernel/ostype
Linux
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- **Layer 3: System Calls**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ strace ls
execve("/usr/bin/ls", ["ls"], 0x7ffdf2d32400 /* 27 vars */) = 0
brk(NULL) = 0x57ffae738000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7813887bd000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=19955, ...}) = 0
mmap(NULL, 19955, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7813887b8000
close(3) = 0
```

- **Layer 4: System Programs**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ps aux
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 1 0.0 0.1 22116 12016 ? Ss 20:37 0:09 /sbin/init
root 2 0.0 0.0 3072 1664 ? Sl 20:37 0:00 /init
root 7 0.0 0.0 3072 1792 ? Sl 20:37 0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root 47 1.1 0.1 66020 10992 ? S-s 20:37 2:05 /usr/lib/systemd/systemd-journald
root 94 0.0 0.0 25108 6144 ? Ss 20:37 0:00 /usr/lib/systemd/systemd-udevd
systemd+ 109 0.0 0.1 21456 12032 ? Ss 20:37 0:01 /usr/lib/systemd/systemd-resolved
systemd+ 117 0.0 0.0 91024 7552 ? Ssl 20:37 0:02 /usr/lib/systemd/systemd-timesyncd
root 168 0.0 0.0 4236 2688 ? Ss 20:37 0:00 /usr/sbin/cron -f -P
message+ 169 0.0 0.0 9636 4864 ? Ss 20:37 0:04 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root 183 0.0 0.1 17968 8320 ? Ss 20:37 0:01 /usr/lib/systemd/systemd-logind
root 186 0.0 0.1 1756096 13184 ? Ssl 20:37 0:00 /usr/libexec/wnsl-pro-service -vv
syslog 200 0.0 0.0 222508 5632 ? Ssl 20:37 0:00 /usr/sbin/rsyslogd -n -iNONE
root 205 0.0 0.0 3160 1920 hvcB Ss+ 20:37 0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
root 208 0.0 0.0 3116 1792 tty1 Ss+ 20:37 0:00 /sbin/agetty -o -p -- \u --noclear - linux
root 214 0.0 0.2 107032 22144 ? Ssl 20:37 0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root 286 0.0 0.0 3076 1024 ? Ss 20:37 0:00 /init
root 288 0.0 0.0 3092 1028 ? S 20:37 0:00 /init
asus 296 0.0 0.0 6068 5208 pts/0 Ss 20:37 0:00 -bash
root 298 0.0 0.0 6824 3960 pts/1 Ss 20:37 0:00 /bin/login -f
asus 359 0.0 0.1 203112 11008 ? Ss 20:37 0:00 /usr/lib/systemd/systemd --user
asus 360 0.0 0.0 21148 3516 ? S 20:37 0:00 (sd-pam)
asus 386 0.0 0.0 6072 4736 pts/1 S+ 20:37 0:00 -bash
asus 1197 0.0 0.0 26800 1536 pts/0 T 21:06 0:00 ./simple_shell
asus 5029 100 0.0 8280 4096 pts/0 R+ 23:42 0:00 ps aux
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

## 2.8.2 Microkernel vs Monolithic

- **Lihat ukuran kernel (Monolithic)**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ ls -lh /boot/vmlinuz-$(uname -r)
ls: cannot access '/boot/vmlinuz-6.6.87.2-microsoft-standard-WSL2': No such file or directory
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- **Lihat built-in vs module**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ cat /boot/config-$(uname -r) | grep "=y" | wc -l # Built-in
cat: /boot/config-6.6.87.2-microsoft-standard-WSL2: No such file or directory
0
asus@LAPTOP-VRS206DG:/tmp/test_dir$ cat /boot/config-$(uname -r) | grep "=m" | wc -l # Modules
cat: /boot/config-6.6.87.2-microsoft-standard-WSL2: No such file or directory
0
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

- **Lihat loaded modules**

```
asus@LAPTOP-VRS206DG:/tmp/test_dir$ lsmod | wc -l
19
asus@LAPTOP-VRS206DG:/tmp/test_dir$ |
```

## 2.8.3 Program - Virtual File System

```
asus@LAPTOP-VRS206DG:~$ nano procfs_read.c
asus@LAPTOP-VRS206DG:~$ gcc -o procfs_read procfs_read.c
asus@LAPTOP-VRS206DG:~$./procfs_read

=== CPU Information ===
processor : 0
vendor_id : AuthenticAMD
cpu family : 23
model : 160
model name : AMD Ryzen 5 7520U with Radeon Graphics
stepping : 0
microcode : 0xffffffff
cpu MHz : 2794.547
cache size : 512 KB
physical id : 0

=== Memory Information ===
MemTotal: 7758048 kB
MemFree: 6925292 kB
MemAvailable: 7251824 kB
Buffers: 402880 kB
Cached: 489136 kB

=== Process Information ===
Name: procfs_read
task: 0022
State: R (running)
Tgid: 528
Ngid: 0
Pid: 528
PPid: 341
TracerPid: 0
Uid: 1000 1000 1000 1000
Gid: 1000 1000 1000 1000
asus@LAPTOP-VRS206DG:~$ |
```

## 2.9 Building and Booting an Operating System

Pada praktik ini, mahasiswa mempelajari bagaimana sistem operasi Linux dibangun dan dijalankan melalui tahapan proses booting serta pengaturan kernel parameters. Proses booting merupakan langkah awal ketika komputer dinyalakan, dimulai dari BIOS/UEFI, dilanjutkan ke bootloader (GRUB), lalu memuat kernel dan sistem utama hingga siap digunakan.

Melalui perintah `dmesg | less`, mahasiswa dapat melihat pesan kernel saat sistem melakukan inisialisasi perangkat keras. Kemudian, dengan `systemd-analyze`, `systemd-analyze blame`, dan `systemd-analyze critical-chain`, mahasiswa dapat menganalisis waktu yang dibutuhkan setiap layanan untuk memulai, serta mengidentifikasi proses yang paling kritis dalam booting. File konfigurasi GRUB (`/boot/grub/grub.cfg`) memperlihatkan urutan dan parameter yang digunakan saat sistem dijalankan, sedangkan `lsinitramfs` menampilkan isi dari *initial RAM filesystem* (*initramfs*) yang berisi driver dan skrip penting untuk menyiapkan sistem sebelum root filesystem dimuat.

Pada praktik Kernel Parameters, mahasiswa mempelajari cara membaca dan mengubah parameter kernel yang digunakan saat boot. Perintah `cat /proc/cmdline` menunjukkan parameter aktif, sedangkan untuk mengubahnya dapat dilakukan secara sementara (dengan menekan `e` pada menu GRUB saat boot) atau permanen dengan mengedit file `/etc/default/grub` dan menjalankan `sudo update-grub`.

Praktik ini membantu mahasiswa memahami urutan kerja sistem operasi dari awal komputer menyala hingga siap digunakan, serta bagaimana pengaturan kernel dapat memengaruhi performa dan perilaku sistem secara keseluruhan.

### 2.9.1 Analisis Boot Process

- **Lihat boot messages**

```

asus@LAPTOP-VRS206DG:~$ dmesg | less
[1]+ Stopped dmesg | less
asus@LAPTOP-VRS206DG:~$ |

```

```
[0.000000] Linux version 6.6.87.2-microsoft-standard-WSL2 (root@439a258ad544) (gcc (GCC) 11.2.0, GNU ld (GNU Binutils) 2.37) #1 SMP PREEMPT_DYNAMIC Thu
Jun 5 18:30:46 UTC 2025
[0.000000] Command line: initrd=initrd.img WSL_ROOT_INIT=1 panic=1 nr_cpus=8 hv_utils.timesync_implicit=1 console=hvc0 debug ptty.legacy_count=0 WSL_EN
ABLE_CRASH_DUMP=1
[0.000000] KERNEL supported cpus:
[0.000000] Intel GenuineIntel
[0.000000] AMD AuthenticAMD
[0.000000] BIOS-provided physical RAM map:
[0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009ffff] usable
[0.000000] BIOS-e820: [mem 0x00000000000e0000-0x00000000000e0fff] reserved
[0.000000] BIOS-e820: [mem 0x0000000001000000-0x00000000001fffff] ACPI data
[0.000000] BIOS-e820: [mem 0x0000000000200000-0x0000000000f7ffff] usable
[0.000000] BIOS-e820: [mem 0x0000000100000000-0x00000001f0dfffff] usable
[0.000000] NX (Execute Disable) protection: active
[0.000000] APIC: Static calls initialized
[0.000000] DMI not present or invalid.
[0.000000] Hypervisor detected: Microsoft Hyper-V
[0.000000] Hyper-V: privilege flags low 0xae7f, high 0x3b8030, hints 0x900c2c, misc 0xe0bed7b6
[0.000000] Hyper-V: Nested features: 0x4a0000
[0.000000] Hyper-V: LAPIC Timer Frequency: 0xc3500
[0.000000] Hyper-V: Using hypercall for remote TLB flush
[0.000000] clocksource: hyperv_clocksource_tsc_page: mask: 0xffffffffffffff max_cycles: 0x24e6a1710, max_idle_ns: 440795202120 ns
[0.000000] clocksource: hyperv_clocksource_msr: mask: 0xffffffffffffff max_cycles: 0x24e6a1710, max_idle_ns: 440795202120 ns
[0.000000] tsc: Detected 2794.547 MHz processor
[0.000076] e820: update [mem 0x00000000-0x00000fff] usable ==> reserved
[0.000081] e820: remove [mem 0x000a0000-0x000fffff] usable
[0.000084] last_pfn = 0x1f0e00 max_arch_pfn = 0x400000000
[0.000112] MTRR map: 5 entries (4 fixed + 1 variable; max 20), built from 8 variable MTRRs
[0.000115] x86/PAT: Configuration [0-7]: WB UC UC UC WB UC- WT
[0.000192] last_pfn = 0xf0000 max_arch_pfn = 0x400000000
[0.000208] Using GB pages for direct mapping
[0.000320] RAMDISK: [mem 0x04be4000-0x04e7ffff]
[0.000323] ACPI: Early table checksum verification disabled
[0.000327] ACPI: RSDP 0x00000000000E0000 000024 (v02 VIRTUAL)
[0.000331] ACPI: XSDT 0x0000000000100000 000044 (v01 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
[0.000337] ACPI: FACP 0x0000000000101000 000114 (v06 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
[0.000342] ACPI: DSDT 0x00000000001011B8 01E11C (v02 MSFTVM DSDT01 00000001 INTL 20230628)
[0.000345] ACPI: FACS 0x0000000000101114 000040
:]
```

- **Lihat systemd boot analysis**

```
asus@LAPTOP-VRS206DG:~$ systemd-analyze
Startup finished in 5.688s (userspace)
graphical.target reached after 5.566s in userspace.
asus@LAPTOP-VRS206DG:~$ systemd-analyze blame
2.264s landscape-client.service
2.213s systemd-resolved.service
1.702s systemd-timesyncd.service
1.030s snapd.seeded.service
841ms snapd.service
658ms dev-sdd.device
622ms wsl-pro.service
327ms rsyslog.service
279ms user@1000.service
279ms dpkg-db-backup.service
256ms systemd-udev-trigger.service
222ms systemd-logind.service
102ms c2scrub_reap.service
108ms systemd-udevd.service
141ms systemd-journal-flush.service
121ms dbus.service
113ms keyboard-setup.service
111ms systemd-journald.service
99ms systemd-tapfiles-setup.service
90ms systemd-sysctl.service
90ms setttrg.service
88ms systemd-tapfiles-setup-dev-early.service
76ms dev-hugepages.mount
75ms dev-mqueue.mount
73ms sys-kernel-debug.mount
71ms sys-kernel-tracing.mount
57ms systemd-binfmt.service
55ms mod-static-modules.service
53ms modprobe@configs.service
51ms modprobe@dm_mod.service
49ms modprobe@drm.service
45ms systemd-update-utp.service
42ms modprobe@efi_ostore.service
42ms user-runtime-dir@1000.service
41ms snapd.socket
40ms modprobe@fuse.service
39ms modprobe@loop.service
39ms systemd-user-sessions.service
lines 1-30
^
asus@LAPTOP-VRS206DG:~$ systemd-analyze critical-chain
The time when unit became active or started is printed after the "@" character.
The time the unit took to start is printed after the "+" character.

graphical.target @5.566s
├─multi-user.target @5.559s
│ └─snapd.seeded.service @3.321s +1.030s
│ └─basic.target @3.249s
│ └─sockets.target @3.248s
│ └─snapd.socket @3.199s +41ms
│ └─sysinit.target @3.178s
│ └─systemd-resolved.service @963ms +2.213s
│ └─systemd-tapfiles-setup.service @630ms +99ms
│ └─systemd-journal-flush.service @634ms +141ms
│ └─systemd-journald.service @459ms +111ms
│ └─systemd-journald.socket @426ms
│ └─system.slice @388ms
│ └─.slice @387ms
asus@LAPTOP-VRS206DG:~$ |
```

- **Lihat GRUB configuration**

```
asus@LAPTOP-VRS206DG:~$ cat /boot/grub/grub.cfg | head -50
cat: /boot/grub/grub.cfg: No such file or directory
asus@LAPTOP-VRS206DG:~$ |
```

- **Lihat initramfs**

```

asus@LAPTOP-VRS206DG:~$ lsinitramfs /boot/initrd.img-$(uname -r) | head -20
/usr/bin/unmkinitramfs: 64: cannot open /boot/initrd.img-6.6.87.2-microsoft-standard-WSL2: No such file
/usr/bin/unmkinitramfs: 57: cannot open /boot/initrd.img-6.6.87.2-microsoft-standard-WSL2: No such file
/usr/bin/unmkinitramfs: 38: cannot open /boot/initrd.img-6.6.87.2-microsoft-standard-WSL2: No such file
cpio: premature end of archive
asus@LAPTOP-VRS206DG:~$

```

## 2.10 Operating-System Debugging

Praktik Operating-System Debugging pada Linux bertujuan untuk memahami cara menemukan dan menganalisis kesalahan dalam sistem maupun program. Dengan strace, pengguna dapat melacak system calls yang dijalankan oleh perintah seperti ls, serta melihat berapa lama setiap system call berlangsung. Sementara itu, ltrace digunakan untuk memantau panggilan library functions agar programmer mengetahui fungsi-fungsi pustaka apa saja yang dijalankan. GDB (GNU Debugger) berfungsi mendeteksi kesalahan logika atau bug dalam program, seperti pada contoh buggy.c yang memiliki kesalahan pada batas array. dmesg digunakan untuk membaca log kernel, memantau error, atau peringatan sistem. Selain itu, Valgrind membantu mendeteksi kebocoran memori, sedangkan perf digunakan untuk menganalisis performa sistem dan memonitor aktivitas CPU. Semua alat ini memungkinkan pengguna memahami perilaku sistem operasi secara mendalam dan memperbaiki masalah dengan efisien.

### 2.10.1 Using strace

- **Trace system calls**

[illegible]



- [illegible]

- ```

asus@LAPTOP-VR5206DG:~$ strace -c ls
buggy      file_monitor.c      info_syscalls      log.txt      os_services.c:      process_syscalls      process_syscalls.c: save      simple_shell      test.file.txt
buggy.c    file_syscalls        info_syscalls.c    os_services  process_menanger  process_syscalls.c    procfs_read      simple_shell.c    trace.txt
file_monitor file_syscalls.c      info_syscalls.c    os_services.c process_menanger.c process_syscalls.c: procfs_read.c     syscall_test.txt
% time      seconds      usecs/call      calls      errors      syscall
-----
24.15      0.002273      64      35      13      openat
18.56      0.001747      58      30      0      mmap
13.21      0.001243      54      23      0      fstat
11.68      0.001099      45      24      0      close
8.55       0.000805      805      1      0      execve
7.04       0.000663      66      10      0      read
3.03       0.000405      95      3      0      write
2.06       0.000269      53      5      0      mprotect
1.50       0.000101      70      2      0      getdents64
1.37       0.000129      64      2      0      ioctl
1.31       0.000123      41      3      0      brk
1.13       0.000106      53      2      1      statfs
1.09       0.000103      51      2      2      access
1.00       0.000094      47      2      0      pread64
0.63       0.000059      59      1      0      munmap
0.56       0.000053      53      1      0      getrandom
0.40       0.000038      38      1      0      arch_prctl
0.40       0.000038      38      1      0      futex
0.38       0.000036      36      1      0      set_tid_address
0.38       0.000036      36      1      0      set_robust_list
0.38       0.000036      36      1      0      prlimit64
0.37       0.000035      35      1      0      rseq
-----
100.00      0.009411      61      152      16      total
asus@LAPTOP-VR5206DG:~$

```


- ```

root@LAPTOP-VRS206DQ:~# strace -o trace.txt ls
buggy file_monitor.c info_syscalls.c log.txt os_services.c process_syscalls process_syscalls.c.save simple_shell test_file.txt
buggy.c file_syscalls info_syscalls.c os_services process_manager process_syscalls.c procfs_read simple_shell.c trace.txt
file_monit file_syscalls.c info_syscalls.c os_services.c process_manager.c process_syscalls.c procfs_read.c syscall_test.txt

root@LAPTOP-VRS206DQ:~$ cat trace.txt
execve("/usr/bin/Ls", ["ls"], 0x7ffeb4123bb0 /* 27 vars */) = 0
brk(NULL) = 0x61e5f14e4800
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7a49f263b000
openat(AT_FDCWD, "/etc/ld.so.preload", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=19955, ...}) = 0
mmap(NULL, 19955, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7a49f263b000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0... 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=174472, ...}) = 0
mmap(NULL, 181960, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7a49f2609000
mmap(0x7a49f260f000, 118784, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x6000) = 0x7a49f260f000
mmap(0x7a49f262c000, 24576, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x23000) = 0x7a49f262c000
mmap(0x7a49f2632000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x29000) = 0x7a49f2632000
mmap(0x7a49f2634000, 5832, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7a49f2634000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0... 832) = 832
pread64(3, {"\6\0\0\0\4\0... 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, {"\6\0\0\0\4\0... 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7a49f2200000
mmap(0x7a49f2228000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7a49f2228000
mmap(0x7a49f23b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7a49f23b0000
mmap(0x7a49f23ff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7a49f23ff000
mmap(0x7a49f24b5000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7a49f24b5000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcr2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0... 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=628344, ...}) = 0
mmap(NULL, 627072, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7a49f256f000
mmap(0x7a49f2571000, 405060, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) = 0x7a49f2571000
mmap(0x7a49f25df000, 163840, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x70000) = 0x7a49f25df000
mmap(0x7a49f2607000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x97000) = 0x7a49f2607000

```

- **Trace library calls**

- ```

sus@LAPTOP-VRS206DG:~$ ltrace ls
strchr("ls", '/') = nil
setlocale(LC_ALL, "") = "C.UTF-8"
bindtextdomain("coreutils", "/usr/share/locale") = "/usr/share/locale"
textdomain("coreutils") = "coreutils"
__cxa_atexit(0x5a348acd8880, 0, 0x5a348acf0008, 0) = 0
getopt_long(1, 0x7ffffac753a8, "abcdcfghiklmnopqrstuvw:ABCDEFGH:I:...", 0x5a348acef340, -1) = -1
getenv("LS_BLOCK_SIZE") = nil
getenv("BLOCK_SIZE") = nil
getenv("BLOCKSIZE") = nil
getenv("POSIXLY_CORRECT") = nil
getenv("BLOCK_SIZE") = nil
isatty(1) = 1
ioctl(1, 21523, 0x7ffffac751c0) = 0
getenv("TABSIZE") = nil
getenv("QUOTING_STYLE") = nil
__errno_location() = 0x7c3a93a726a0
__errno_location() = 0x7c3a93a726a0
getenv("TZ") = nil
reallocarray(0, 100, 208, 0x5a34989166c0) = 0x5a34989166d0
strlen(".") = 1
__memcpy_chk(0x5a349891b850, 0x5a348ace7edc, 2, 2) = 0x5a349891b850
__errno_location() = 0x7c3a93a726a0
opendir(".") = 0x5a349891b870
readdir(0x5a349891b870) = 0x5a349891b8a0
__errno_location() = 0x7c3a93a726a0
__ctype_get_mb_cur_max() = 6
strlen("info_syscalls.c") = 15
strlen("info_syscalls.c") = 15
__memcpy_chk(0x5a34989238b0, 0x5a349891b8b3, 16, 16) = 0x5a34989238b0
readdir(0x5a349891b870) = 0x5a349891b8c8
__errno_location() = 0x7c3a93a726a0
__ctype_get_mb_cur_max() = 6
strlen("process_syscalls.c:.save") = 24
strlen("process_syscalls.c:.save") = 24
__memcpy_chk(0x5a34989238d0, 0x5a349891b8db, 25, 25) = 0x5a34989238d0
readdir(0x5a349891b870) = 0x5a349891b8f8
readdir(0x5a349891b870) = 0x5a349891b910
readdir(0x5a349891b870) = 0x5a349891b940

```

- Dengan detail

```
asus@LAPTOP-VRS206DG:~$ ltrace -e ls
buggy      file_monitor.c  info_syscalls  log.txt      os_services.c  process_syscalls  process_syscalls.c:.save  simple_shell  test_file.txt
buggy.c    file_syscalls   info_syscalls.c  os_services  process_menanger  process_syscalls.c  procfs_read  simple_shell.c  trace.txt
file_monitor  file_syscalls.c  info_syscalls.c  os_services.c  process_menanger  process_syscalls.c  procfs_read.c  syscall_test.txt

% time      seconds  usecs/call      calls      function
-----
18.04      0.043167      353          122  __errno_location
15.46      0.036984      352          105  strlen
13.92      0.033294      358           93  strcoll
11.94      0.028575      375           76  __overflow
11.10      0.026552      340           78  __ctype_get_mb_cur_max
 7.98      0.019094     19094           1  setlocale
 5.92      0.014167      363           39  readdir
 3.99      0.009541      366           26  fwrite_unlocked
 3.72      0.009904      329           27  __memcpy_chk
 1.33      0.003193      319           10  memcpy
 1.21      0.002887     1443           2  fflush
 1.11      0.002658      332           8  getenv
 0.63      0.001509      377           4  __freading
 0.53      0.001278      639           2  fclose
 0.52      0.001255      313           4  reallocarray
 0.35      0.000832      416           2  __fpending
 0.32      0.000759      379           2  fileno
 0.30      0.000709      709           1  opendir
 0.26      0.000627      627           1  ioctl
 0.26      0.000611      611           1  isatty
 0.23      0.000545      545           1  strrchr
 0.21      0.000508      508           1  closedir
 0.16      0.000386      386           1  setjmp
 0.14      0.000343      343           1  bindtextdomain
 0.13      0.000302      302           1  getopt_long
 0.12      0.000292      292           1  __cxa_atexit
 0.11      0.000259      259           1  textdomain

-----
100.00      0.239231      611 total
asus@LAPTOP-VRS206DG:~$
```

2.10.3 Using GDB

```
asus@LAPTOP-VRS206DG:~$ nano buggy.c
asus@LAPTOP-VRS206DG:~$
```

- Compile dengan debug symbols

```
asus@LAPTOP-VRS206DG:~$ gcc -g -o buggy buggy.c
asus@LAPTOP-VRS206DG:~$
```

- Jalankan dengan GDB

```
asus@LAPTOP-VRS206DG:~$ gdb ./buggy
GNU gdb (Ubuntu 15.0.50.20240403-bubuntal) 15.0.50.20240403-gi
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./buggy...
(gdb) break main
Breakpoint 1 at 0x1238: file buggy.c, line 17.
(gdb) run
Starting program: /home/asus/buggy

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7c3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at buggy.c:17
17      buggy_function(6);
(gdb) next
array[0] = 0
array[1] = 2
array[2] = 4
array[3] = 6
array[4] = 8
18      return 0;
(gdb) stop
19      }
(gdb) print n
No symbol "n" in current context.
(gdb) continue
Continuing.
[Inferior 1 (process 633) exited normally]
(gdb) backtrace
No stack.
(gdb) quit
asus@LAPTOP-VRS206DG:~$
```

2.10.4 Kernel Debugging dengan dmesg

- Lihat kernel log

```
asus@LAPTOP-VRS2960G:~$ dmesg
0.000000] Linux version 6.6.87.2-microsoft-standard-WSL2 (root@439a258ad544) (gcc (GCC) 11.2.0, GNU ld (GNU Binutils) 2.37) #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025
0.000000] Command line: initrd=\initrd.img WSL_ROOT_INIT=1 panic=1 nr_cpus=8 hv_utils.timesync_implicit=1 console=hvc0 debug ptty,legacy_count=0 WSL_ENABLE_CRASH_DUMP=1
0.000000] KERNEL supported cpus:
0.000000]   Intel GenuineIntel
0.000000]   AMD AuthenticAMD
0.000000] BIOS-provided physical RAM map:
0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009ffff] usable
0.000000] BIOS-e820: [mem 0x00000000000a0000-0x00000000000e0fff] reserved
0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000001fffff] ACPI data
0.000000] BIOS-e820: [mem 0x0000000000200000-0x00000000007fffff] usable
0.000000] BIOS-e820: [mem 0x0000000010000000-0x000000001fd0ffff] usable
0.000000] NX (Execute Disable) protection: active
0.000000] APIC: Static calls initialized
0.000000] DMI not present or invalid.
0.000000] Hypervisor detected: Microsoft Hyper-V
0.000000] Hyper-V: privilege flags low 0xae7f, high 0x3b8030, hints 0x900c2c, misc 0xe0bed7b6
0.000000] Hyper-V: Nested features: 0x4a0000
0.000000] Hyper-V: LAPIC Timer Frequency: 0xc3500
0.000000] Hyper-V: Using hypercall for remote TLB flush
0.000000] clocksource: hyperv_clocksource_tsc_page: mask: 0xffffffffffffff max_cycles: 0x24e6a1710, max_idle_ns: 440795202120 ns
0.000000] clocksource: hyperv_clocksource_msr: mask: 0xffffffffffffff max_cycles: 0x24e6a1710, max_idle_ns: 440795202120 ns
0.000000] tsc: Detected 2794.547 MHz processor
0.000076] e820: update [mem 0x00000000-0x0000ffff] usable ==> reserved
0.000081] e820: remove [mem 0x000a0000-0x000fffff] usable
0.000084] last_pfn = 0x1f0e00 max_arch_pfn = 0x400000000
0.000112] MTRR map: 5 entries (4 fixed + 1 variable; max 20), built from 8 variable MTRRs
0.000115] x86/PAT: Configuration [0-7]: WB WC UC- UC WB WP UC- WT
0.000192] last_pfn = 0xf8000 max_arch_pfn = 0x400000000
0.000208] Using GB pages for direct mapping
0.000320] RAMDISK: [mem 0x04be4000-0x04e7ffff]
0.000323] ACPI: Early table checksum verification disabled
0.000327] ACPI: RSDP 0x0000000000000000 000024 (v02 VIRTUAL)
0.000331] ACPI: XSDT 0x0000000000100000 000044 (v01 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000337] ACPI: FACP 0x0000000000101000 000114 (v06 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000342] ACPI: DSDT 0x00000000001011B8 01E11C (v02 MSFTVM DSDT01 00000001 INTL 20230628)
0.000345] ACPI: FACS 0x0000000000101114 000040
0.000347] ACPI: OEM0 0x0000000000101154 000064 (v01 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000350] ACPI: SRAT 0x000000000011F204 000330 (v02 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000353] ACPI: APIC 0x000000000011F604 000088 (v04 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000355] ACPI: Reserving FACP table memory at [mem 0x101000-0x101133]
0.000356] ACPI: Reserving DSDT table memory at [mem 0x1011B8-0x11f2d3]
0.000357] ACPI: Reserving FACS table memory at [mem 0x101114-0x101153]
0.000358] ACPI: Reserving OEM0 table memory at [mem 0x101154-0x1011b7]
0.000359] ACPI: Reserving SRAT table memory at [mem 0x11f2d4-0x11f603]
0.000360] ACPI: Reserving APIC table memory at [mem 0x11f604-0x11f68b]
0.000396] SRAT: PXM 0 -> APIC 0x00 -> Node 0
0.000398] SRAT: PXM 0 -> APIC 0x01 -> Node 0
0.000399] SRAT: PXM 0 -> APIC 0x02 -> Node 0
0.000399] SRAT: PXM 0 -> APIC 0x03 -> Node 0
0.000400] SRAT: PXM 0 -> APIC 0x04 -> Node 0
0.000401] SRAT: PXM 0 -> APIC 0x05 -> Node 0
0.000401] SRAT: PXM 0 -> APIC 0x06 -> Node 0
0.000402] SRAT: PXM 0 -> APIC 0x07 -> Node 0
0.000404] ACPI: SRAT: Node 0 PXM 0 [mem 0x00000000-0xf7ffff] hotplug
```

- Follow kernel log

```
asus@LAPTOP-VRS2960G:~$ dmesg -w
0.000000] Linux version 6.6.87.2-microsoft-standard-WSL2 (root@439a258ad544) (gcc (GCC) 11.2.0, GNU ld (GNU Binutils) 2.37) #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025
0.000000] Command line: initrd=\initrd.img WSL_ROOT_INIT=1 panic=1 nr_cpus=8 hv_utils.timesync_implicit=1 console=hvc0 debug ptty,legacy_count=0 WSL_ENABLE_CRASH_DUMP=1
0.000000] KERNEL supported cpus:
0.000000]   Intel GenuineIntel
0.000000]   AMD AuthenticAMD
0.000000] BIOS-provided physical RAM map:
0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009ffff] usable
0.000000] BIOS-e820: [mem 0x00000000000a0000-0x00000000000e0fff] reserved
0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000001fffff] ACPI data
0.000000] BIOS-e820: [mem 0x0000000000200000-0x00000000007fffff] usable
0.000000] BIOS-e820: [mem 0x0000000010000000-0x000000001fd0ffff] usable
0.000000] NX (Execute Disable) protection: active
0.000000] APIC: Static calls initialized
0.000000] DMI not present or invalid.
0.000000] Hypervisor detected: Microsoft Hyper-V
0.000000] Hyper-V: privilege flags low 0xae7f, high 0x3b8030, hints 0x900c2c, misc 0xe0bed7b6
0.000000] Hyper-V: Nested features: 0x4a0000
0.000000] Hyper-V: LAPIC Timer Frequency: 0xc3500
0.000000] Hyper-V: Using hypercall for remote TLB flush
0.000000] clocksource: hyperv_clocksource_tsc_page: mask: 0xffffffffffffff max_cycles: 0x24e6a1710, max_idle_ns: 440795202120 ns
0.000000] clocksource: hyperv_clocksource_msr: mask: 0xffffffffffffff max_cycles: 0x24e6a1710, max_idle_ns: 440795202120 ns
0.000000] tsc: Detected 2794.547 MHz processor
0.000076] e820: update [mem 0x00000000-0x0000ffff] usable ==> reserved
0.000081] e820: remove [mem 0x000a0000-0x000fffff] usable
0.000084] last_pfn = 0x1f0e00 max_arch_pfn = 0x400000000
0.000112] MTRR map: 5 entries (4 fixed + 1 variable; max 20), built from 8 variable MTRRs
0.000115] x86/PAT: Configuration [0-7]: WB WC UC- UC WB WP UC- WT
0.000192] last_pfn = 0xf8000 max_arch_pfn = 0x400000000
0.000208] Using GB pages for direct mapping
0.000320] RAMDISK: [mem 0x04be4000-0x04e7ffff]
0.000323] ACPI: Early table checksum verification disabled
0.000327] ACPI: RSDP 0x0000000000000000 000024 (v02 VIRTUAL)
0.000331] ACPI: XSDT 0x0000000000100000 000044 (v01 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000337] ACPI: FACP 0x0000000000101000 000114 (v06 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000342] ACPI: DSDT 0x00000000001011B8 01E11C (v02 MSFTVM DSDT01 00000001 INTL 20230628)
0.000345] ACPI: FACS 0x0000000000101114 000040
0.000347] ACPI: OEM0 0x0000000000101154 000064 (v01 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000350] ACPI: SRAT 0x000000000011F204 000330 (v02 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000353] ACPI: APIC 0x000000000011F604 000088 (v04 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
0.000355] ACPI: Reserving FACP table memory at [mem 0x101000-0x101133]
0.000356] ACPI: Reserving DSDT table memory at [mem 0x1011B8-0x11f2d3]
0.000357] ACPI: Reserving FACS table memory at [mem 0x101114-0x101153]
0.000358] ACPI: Reserving OEM0 table memory at [mem 0x101154-0x1011b7]
0.000359] ACPI: Reserving SRAT table memory at [mem 0x11f2d4-0x11f603]
0.000360] ACPI: Reserving APIC table memory at [mem 0x11f604-0x11f68b]
0.000396] SRAT: PXM 0 -> APIC 0x00 -> Node 0
0.000398] SRAT: PXM 0 -> APIC 0x01 -> Node 0
0.000399] SRAT: PXM 0 -> APIC 0x02 -> Node 0
0.000399] SRAT: PXM 0 -> APIC 0x03 -> Node 0
0.000400] SRAT: PXM 0 -> APIC 0x04 -> Node 0
0.000401] SRAT: PXM 0 -> APIC 0x05 -> Node 0
0.000401] SRAT: PXM 0 -> APIC 0x06 -> Node 0
0.000402] SRAT: PXM 0 -> APIC 0x07 -> Node 0
```

- Filter by level

```
asus@LAPTOP-VRS206DG:~$ dmesg -l err
[ 0.190553] PCI: Fatal: No config space access function found
[ 2.752076] misc dxg: dxgk: dxgkio_is_feature_enabled: ioctl failed: -22
[ 2.759404] misc dxg: dxgk: dxgkio_query_adapter_info: ioctl failed: -22
[ 2.769568] misc dxg: dxgk: dxgkio_query_adapter_info: ioctl failed: -22
[ 2.761638] misc dxg: dxgk: dxgkio_query_adapter_info: ioctl failed: -22
[ 2.762752] misc dxg: dxgk: dxgkio_query_adapter_info: ioctl failed: -2
[ 6.381232] WSL (214) ERROR: CheckConnection: getaddrinfo() failed: -5
asus@LAPTOP-VRS206DG:~$
```

```
asus@LAPTOP-VRS206DG:~$ dmesg -l warn
[ 0.137947] Speculative Return Stack OverFlow: IBPB-extending microcode not applied!
[ 0.137948] Speculative Return Stack OverFlow: WARNING: See https://kernel.org/doc/html/latest/admin-guide/hw-vuln/srso.html for mitigation options.
[ 0.226479] PCI: System does not support PCI
[ 0.430092] device-mapper: core: CONFIG_IPTABLE_DISABLE_HTABLE is disabled. Duplicate IMA measurements will not be recorded in the IMA log.
[ 2.305360] pulseaudio[200]: memfd_create() called without MFD_EXEC or MFD_NOEXEC_SEAL set
[ 2.822594] Failed to connect to bus: No such file or directory
[ 3.197158] systemd-journald[44]: File /var/log/journal/4ee66dbc6c004eb186b6f1d288457fde/system.journal corrupted or uncleanly shut down, renaming and replacing.
[ 8.332361] TCP: eth0: Driver has suspect GRO implementation, TCP performance may be compromised.
[ 9.665614] systemd-journald[44]: File /var/log/journal/4ee66dbc6c004eb186b6f1d288457fde/user-1000.journal corrupted or uncleanly shut down, renaming and replacing.
asus@LAPTOP-VRS206DG:~$
```

2.10.5 Memory Leak Detection dengan Valgrind

- Install valgrind

```
asus@LAPTOP-VRS206DG:~$ sudo apt install valgrind
[sudo] password for asus:
Sorry, try again.
[sudo] password for asus:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  valgrind-dbg valgrind-mpi kcachegrind aliothop valkyrie
The following NEW packages will be installed:
  valgrind
0 upgraded, 1 newly installed, 0 to remove and 28 not upgraded.
Need to get 14.9 MB of archives.
After this operation, 78.8 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 valgrind amd64 1:3.22.0-0ubuntu3 [14.9 MB]
Fetched 14.9 MB in 10min 40s (23.3 kB/s)
Selecting previously unselected package valgrind.
(Reading database ... 50585 files and directories currently installed.)
Preparing to unpack .../valgrind_1%3a3.22.0-0ubuntu3_amd64.deb ...
Unpacking valgrind (1:3.22.0-0ubuntu3) ...
Setting up valgrind (1:3.22.0-0ubuntu3) ...
Processing triggers for man-db (2.12.0-4build2) ...
asus@LAPTOP-VRS206DG:~$
```

- Buat program dengan memory leak

```
asus@LAPTOP-VRS206DG:~$ cat > memleak.c << 'EOF'
> #include <stdlib.h>
int main() {
    int *ptr = (int*)malloc(100 * sizeof(int));
    // Forgot to free(ptr)
    return 0;
}
EOF
asus@LAPTOP-VRS206DG:~$ gcc -g -o memleak memleak.c
asus@LAPTOP-VRS206DG:~$
```

- Jalankan dengan valgrind

```
asus@LAPTOP-VRS206DG:~$ valgrind --leak-check=full ./memleak
==846== Memcheck, a memory error detector
==846== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==846== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==846== Command: ./memleak
==846==
==846==
==846== HEAP SUMMARY:
==846==   in use at exit: 400 bytes in 1 blocks
==846==   total heap usage: 1 allocs, 0 frees, 400 bytes allocated
==846==
==846== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==846==   at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==846==   by 0x10915E: main (memleak.c:3)
==846==
==846== LEAK SUMMARY:
==846==   definitely lost: 400 bytes in 1 blocks
==846==   indirectly lost: 0 bytes in 0 blocks
==846==   possibly lost: 0 bytes in 0 blocks
==846==   still reachable: 0 bytes in 0 blocks
==846==   suppressed: 0 bytes in 0 blocks
==846==
==846== For lists of detected and suppressed errors, rerun with: -s
==846== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
asus@LAPTOP-VRS206DG:~$ |
```

2.10.6 Performance Analysis

- CPU profiling dengan perf

```
asus@LAPTOP-VRS206DG:~$ sudo apt install linux-tools-common linux-tools-generic
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
linux-tools-common is already the newest version (6.8.0-85.85).
linux-tools-generic is already the newest version (6.8.0-85.85).
0 upgraded, 0 newly installed, 0 to remove and 28 not upgraded.
asus@LAPTOP-VRS206DG:~$ |
```

- Monitor system calls

```
asus@LAPTOP-VRS206DG:~$ sudo perf trace ls
WARNING: perf not found for kernel 6.6.87.2-microsoft

You may need to install the following packages for this specific kernel:
  linux-tools-6.6.87.2-microsoft-standard-WSL2
  linux-cloud-tools-6.6.87.2-microsoft-standard-WSL2

You may also want to install one of the following packages to keep up to date:
  linux-tools-standard-WSL2
  linux-cloud-tools-standard-WSL2
asus@LAPTOP-VRS206DG:~$ |
```

- **Top-like interface**

```
asus@LAPTOP-VRS206DG:~$ sudo perf trace ls
WARNING: perf not found for kernel 6.6.87.2-microsoft

You may need to install the following packages for this specific kernel:
  linux-tools-6.6.87.2-microsoft-standard-WSL2
  linux-cloud-tools-6.6.87.2-microsoft-standard-WSL2

You may also want to install one of the following packages to keep up to date:
  linux-tools-standard-WSL2
  linux-cloud-tools-standard-WSL2
asus@LAPTOP-VRS206DG:~$ sudo perf top
WARNING: perf not found for kernel 6.6.87.2-microsoft

You may need to install the following packages for this specific kernel:
  linux-tools-6.6.87.2-microsoft-standard-WSL2
  linux-cloud-tools-6.6.87.2-microsoft-standard-WSL2

You may also want to install one of the following packages to keep up to date:
  linux-tools-standard-WSL2
  linux-cloud-tools-standard-WSL2
asus@LAPTOP-VRS206DG:~$ |
```

BAB III

PENUTUP

3.1 KESIMPULAN

Melalui praktikum Sistem Operasi ini, mahasiswa memahami peran penting sistem operasi, khususnya Linux, dalam mengelola sumber daya komputer seperti proses, memori, dan sistem berkas. Praktikum mencakup penerapan layanan dasar OS, penggunaan *system call*, pembuatan *daemon service*, serta analisis struktur internal melalui *kernel module* dan *Virtual File System*. Dari kegiatan ini dapat disimpulkan bahwa sistem operasi berfungsi sebagai penghubung utama antara pengguna dan perangkat keras, serta menjadi fondasi utama bagi jalannya berbagai aplikasi dan layanan dalam sistem komputer secara efisien dan terintegrasi.